1. Construct a 5-to-32-line decoder using 3-to-8-line and 2-to- 4-line decoders. [5 Marks]

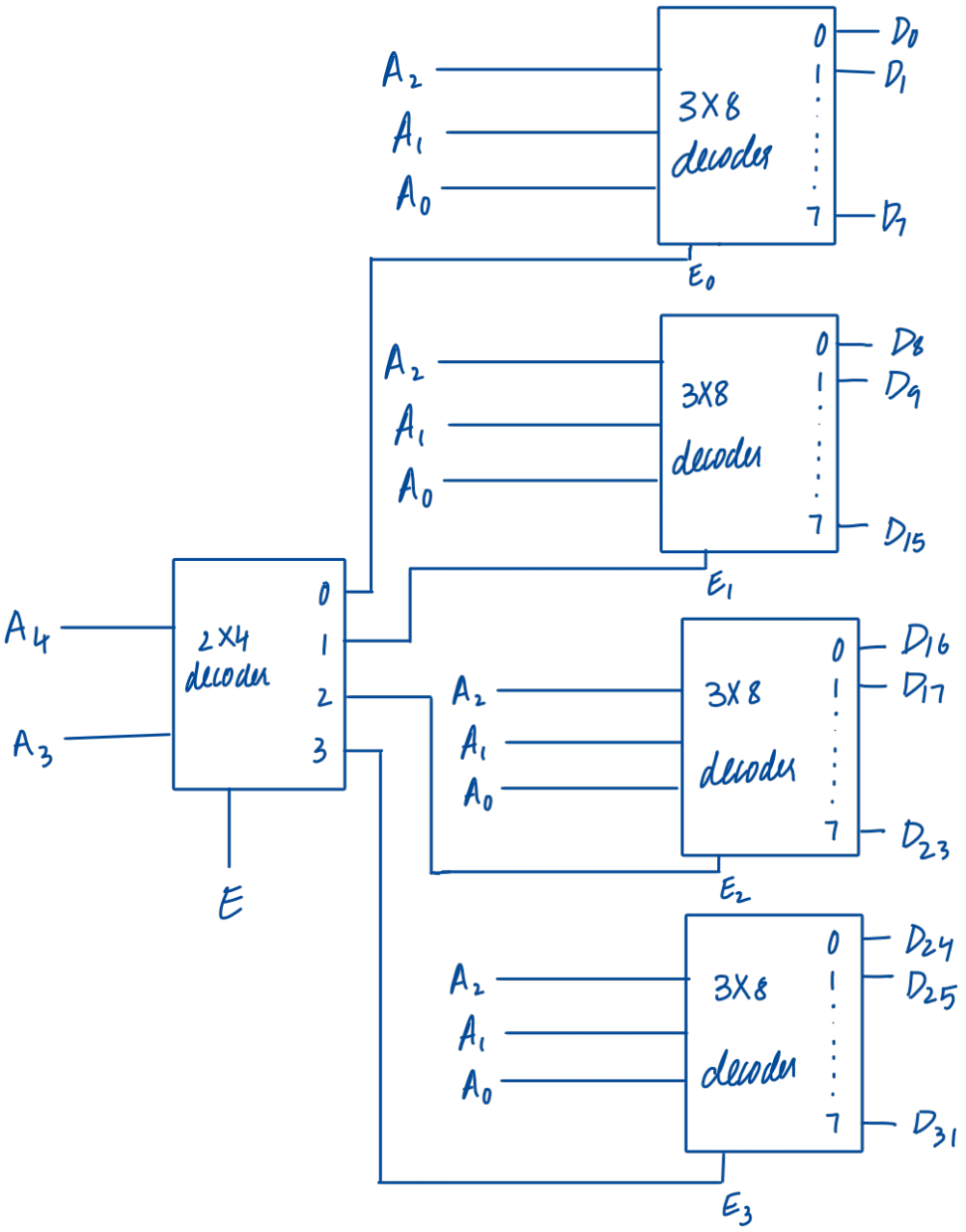
Let the 5 inputs be A_4, A_3, A_2, A_1, A_0
use one 2X4 decoder for the most significant bits (A_4, A_3)

The enable of this decoder acts as the main enable of the 5X32 decoder. Let this be E.
Let the outputs of the 2X4 decoder be E_0, E_1, E_2, E_3
These act as the enables for four 3X4 decoders
The remaining inputs A_2, A_1, A_0 are given as inputs to all four 3X8 decoders

The connections are made as shown:

$A_4 A_3$	Enabled 3X8 decoder	Outputs
00	decoder 0	$D_0 - D_7$
01	decoder 1	$D_8 - D_{15}$
10	decoder 2	$D_{16} - D_{23}$
11	decoder 3	$D_{24} - D_{31}$

Block diagram: (5 marks)



The correct circuit will fetch full marks even if the explanation isn't there

If your solution uses one 3x8 decoders and eight 2x4 decoders, 2.5 marks will be awarded.

2. Implement a Boolean function F(A, B, C) = Σ m(0, 1, 4, 6, 7) [8 Marks]
   a. Using 4x1 MUX. [3 M]
   b. Using decoder. [2 M]
   c. Using minimal number of NAND (& NOT) gates. [3 M]

(a) 4X1 MUX has two select lines ⇒ choose A, B as select lines, then determine what each MUX input should be in terms of C.

let the MUX inputs be I_0, I_1, I_2, I_3

- for AB = 00,
  minterms m_0 and m_1 are both 1
  ⇒ F = 1 ⇒ $\boxed{I_0 = 1}$ → 0.5 marks
- for AB = 01
  minterms m_2 and m_3 are both 0
  ⇒ F = 0 ⇒ $\boxed{I_1 = 0}$ → 0.5 marks
- for AB = 10
  $m_4 = 1$ when $C = 0$
  $m_5 = 0$ when $C = 1$
  ⇒ $F = C'$ ⇒ $\boxed{I_2 = C'}$ → 0.5 marks
- for AB = 11
  minterms m_6 and m_7 are both 1
  ⇒ F = 1 ⇒ $\boxed{I_3 = 1}$ → 0.5 marks

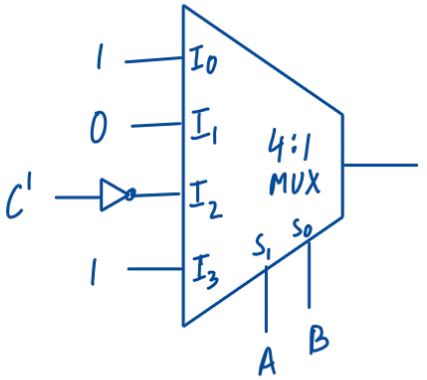
OR

AB	F(when C=0)	F(when C=1)	I_n
00	1	1	$I_0 = 1$
01	0	0	$I_1 = 0$
10	1	0	$I_2 = C'$
11	1	1	$I_3 = 1$

} 2 marks

(Writing either the explanation or the truth table gives 2 marks)

Then, the circuit is :



→ 1 mark

(b) For a 3 variable function, use a 3x8 decoder.

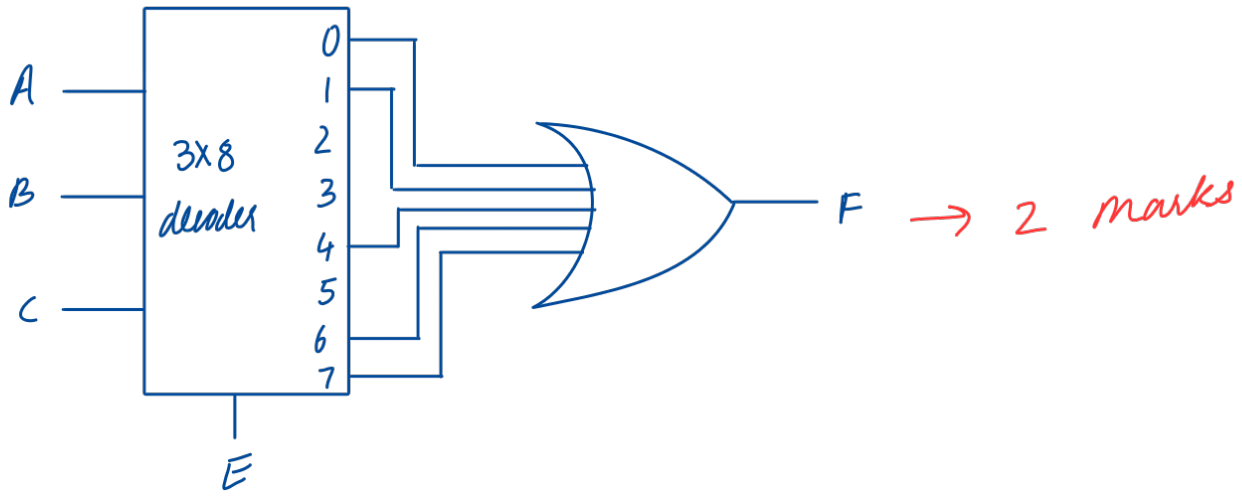
A, B, C are connected to the decoder inputs

The decoder produces outputs $D_0, D_1, \ldots D_7$ where each output corresponds to one minterm

for $F(A,B,C) = \Sigma m(0,1,4,6,7)$, we have:

$$F = D_0 + D_1 + D_4 + D_6 + D_7$$

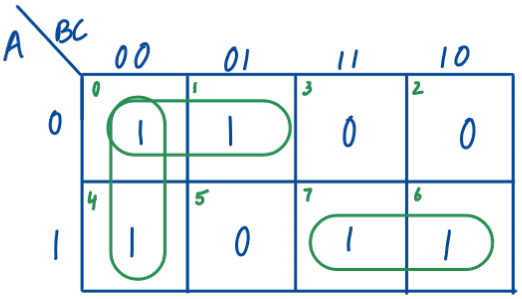
so, connect the decoder outputs to a 5-input OR gate as shown in the circuit :



→ 2 marks

The correct circuit will fetch full marks even if the explanation isn't there

(C) Simplify the Boolean expression using a k-map:



→ 0.5 marks

$\Rightarrow F = B'C' + A'B' + AB$ → 0.5 marks

$\Rightarrow F = AB + B'(A'+C') = AB + B'(AC)'$

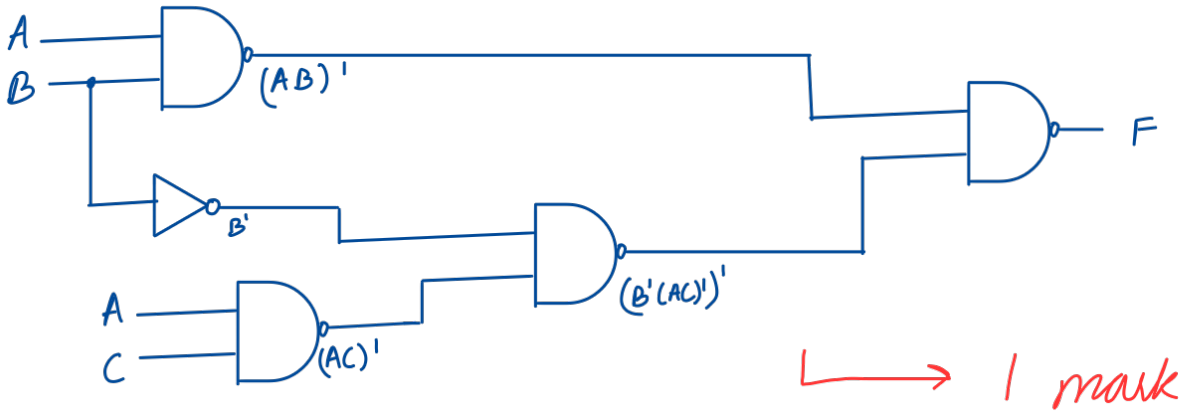
let $X = (AB)'$ and $Y = (AC)' \Rightarrow F = X' + B'Y$

Now, $(F')' = \left((X' + B'Y)'\right)' = \left((X)\cdot(B'Y)'\right)' = \left((AB)'\cdot(B'(AC)')\right)'$ → 0.5 marks

Here,
- 1 NAND is used for $(AB)'$
- 1 NOT is used for B'
- 1 NAND is used for $(AC)'$
- 1 NAND is used for $(B'(AC)')'$
- 1 NAND is used for $((AB)'\cdot(B'(AC)')')'$

} 5 gates in total → 0.5 marks

This can be represented by the circuit:



→ 1 mark

If your final answer uses 6 or 7 gates, and the rest of your solution is correct, 2.5 marks will be awarded for this question.
In all other cases (>7 gates used), 0 marks will be given.

3. Design a combinational circuit that generates the 9's complement of a BCD digit. [5 Marks]

Method - I Using Adders Method Explanation (2 Marks):

- **[1 Marks]:** This subtraction can be performed utilizing a 4-bit binary adder using the 2's complement method.
  - **2's Complement Method:** The subtraction is achieved via $9 + (1\text{'s complement of } X) + 1$. One 4-bit input port of the adder is hardcoded to 9 (1001_2). The BCD input bits are inverted to form the 1's complement, and the initial carry-in (C_0) is hardcoded to logic 1. The adder consequently calculates $9 + \bar{X} + 1$, which strictly evaluates to $9 - X$.

Circuit Diagram (4 Marks):

- The input bits of A3,A2,A1,A0 will be hardcoded to 9(1001) and the input bits of B3,B2,B1,B0 are the input bits .
- They are Xor'ed with M which is set to 1 and the same bit is used as Cin to first adder .
- For Inverting using Not gates will also receive full marks.

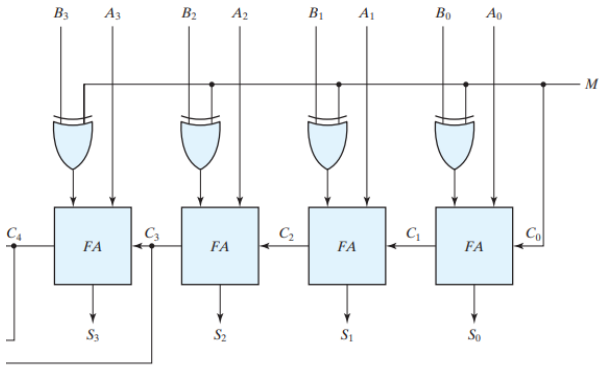


Figure 1: 9's Complement Circuit Diagram

Method -II Using Truth Table

Decimal Digit	BCD Input				9's Complement Output			
	X_3	X_2	X_1	X_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1	0	0	1
1	0	0	0	1	1	0	0	0
2	0	0	1	0	0	1	1	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	0	1
5	0	1	0	1	0	1	0	0
6	0	1	1	0	0	0	1	1
7	0	1	1	1	0	0	1	0
8	1	0	0	0	0	0	0	1
9	1	0	0	1	0	0	0	0
10–15	Dont Care				X	X	X	X

Table 1: Truth table for converting a 4-bit BCD digit to its 9's complement.

Correct Truth Table will receive 1 Mark.

K-Maps and Boolean Expressions (2 Marks):

K-map for Y_3

X_3X_2 \ X_1X_0	00	01	11	10
00	1	1	0	0
01	0	0	0	0
11	X	X	X	X
10	0	0	X	X

K-map for Y_2

X_3X_2 \ X_1X_0	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	X	X	X	X
10	0	0	X	X

K-map for Y_1

X_3X_2 \ X_1X_0	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	X	X	X	X
10	0	0	X	X

K-map for Y_0

X_3X_2 \ X_1X_0	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	X	X	X	X
10	1	0	X	X

Simplified Boolean Expressions:

- $Y_3 = X_3'X_2'X_1'$ *(Red loop in Y_3 map)*
- $Y_2 = X_2'X_1 + X_2X_1' = X_2 \oplus X_1$ *(Blue and Red loops in Y_2 map)*
- $Y_1 = X_1$ *(Red loop in Y_1 map)*
- $Y_0 = X_0'$ *(Red edge-wrap loop in Y_0 map)*

Circuit Diagram (2 Marks):

- **[2 Marks]** Awarded for a correctly drawn logic circuit diagram.
- *Note:* The exact hardware implementation need not be identical to the provided solution. Students are allowed to use different gate combinations as long as they correctly implement the required output functions.

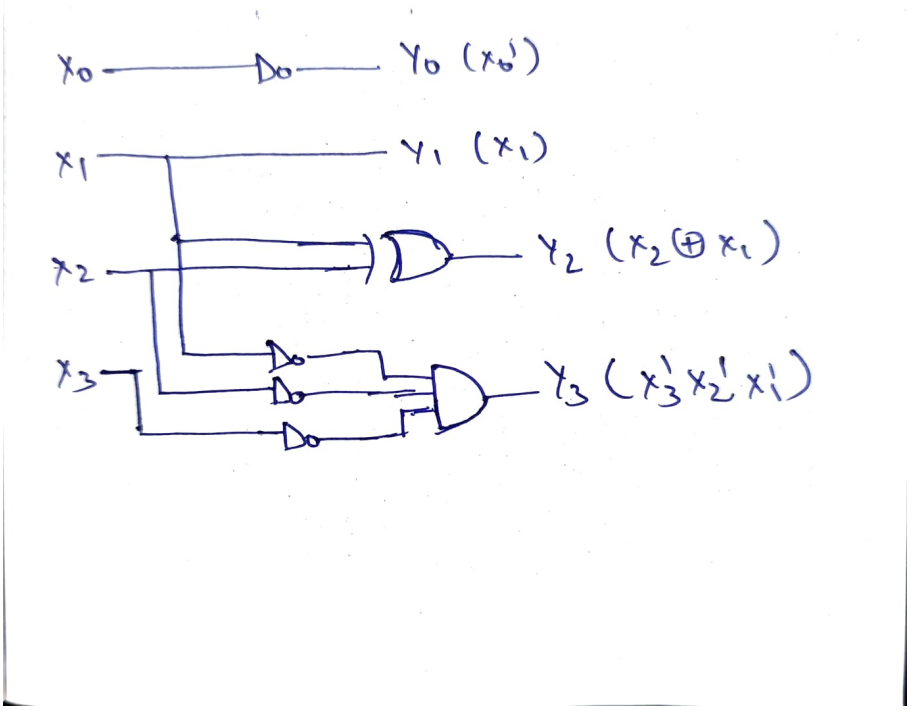


Figure 2: Circuit Diagram

4. (a) What is the need of carry look-a-head adder and how it works? [2 Marks]

Need for CLA (1 Mark):

- In standard ripple carry adders, the carry bit must propagate sequentially through every full adder stage, causing a significant time delay. A Carry Look-Ahead Adder (CLA) is needed to eliminate this cascading ripple delay by calculating carry-in signals in advance, thereby significantly speeding up binary addition.

How it works (1 Mark):

- It computes two intermediate signals at each bit position: the **Generate** signal ($G_i = A_i \cdot B_i$) and the **Propagate** signal ($P_i = A_i \oplus B_i$). Using these signals, the carry logic calculates all carry bits simultaneously using parallel combinational logic (e.g., $C_{i+1} = G_i + P_i C_i$), completely bypassing the sequential wait time of standard adders.

4. (b) What is the advantage of priority encoder? [2 Marks]

Advantage (2 Marks):

- **Handles Input Contention (1 Mark):** A standard encoder produces an incorrect or ambiguous output if multiple inputs are asserted high at the same time. A priority encoder specifically handles the possibility that inputs might be in contention.
- **Priority Resolution (1 Mark):** It resolves this conflict by assigning a strict hardware hierarchy to its inputs. If contention occurs, the logic gates suppress lower-tier signals and only generate the binary output corresponding to the active input with the highest priority.

Q5

a) Treat the 8 physical keys (K_0 to K_7) directly as 8 independent binary input variables, which requires setting up an 8-variable truth table and solving six 8-variable K-maps. [1]

b) An 8-variable K-map contains $2^8 = 256$ cells. Drawing and minimizing 256-cell K-maps manually is highly complex, tedious, and prone to human error. [1]

Method 1: Priority Encoder + Minimized Logic Block

c) [4]

Module 1: 8-to-3 Priority Encoder [1.5]

- **Inputs:** $K_0, K_1, K_2, K_3, K_4, K_5, K_6, K_7$ [0.5]
- **Outputs:** A, B, C [0.5]

[0.5]

K_7	K_6	K_5	K_4	K_3	K_2	K_1	K_0	A	B	C
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	x	0	0	1
0	0	0	0	0	1	x	x	0	1	0
0	0	0	0	1	x	x	x	0	1	1
0	0	0	1	x	x	x	x	1	0	0
0	0	1	x	x	x	x	x	1	0	1
0	1	x	x	x	x	x	x	1	1	0
1	x	x	x	x	x	x	x	1	1	1

Module 2: 3-to-6 Combinatorial Logic Block [2.5]

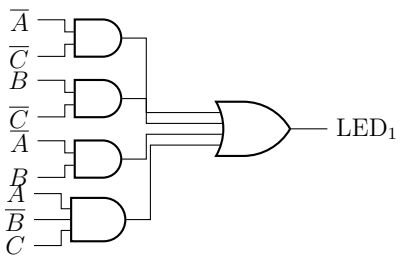
- **Inputs:** A, B, C (from Module 1) [0.5]
- **Outputs:** $\text{LED}_1 \ldots \text{LED}_6$ [0.5]

K-Maps, Equations, and Gate-Level Circuits for Each LED:

- **For LED$_1$:** [0.25]

$A\backslash BC$	00	01	11	10
0	1	0	1	1
1	0	1	0	1

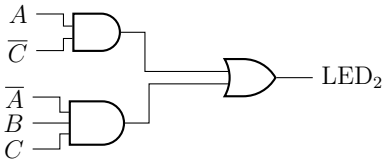
$$\text{LED}_1 = \overline{A}\,\overline{C} + B\overline{C} + \overline{A}B + A\overline{B}C$$



- **For LED$_2$:** [0.25]

$A\backslash BC$	00	01	11	10
0	0	0	1	0
1	1	0	0	1

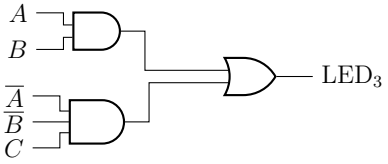
$$\text{LED}_2 = A\overline{C} + \overline{A}BC$$



– **For LED$_3$:** [0.25]

$A\backslash BC$	00	01	11	10
0	0	1	0	0
1	0	0	1	1

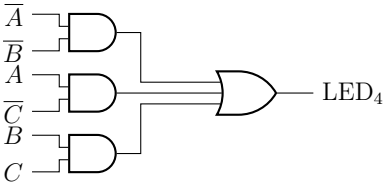
$$\text{LED}_3 = AB + \overline{A}\,\overline{B}C$$



– **For LED$_4$:** [0.25]

$A\backslash BC$	00	01	11	10
0	1	1	1	0
1	1	0	1	1

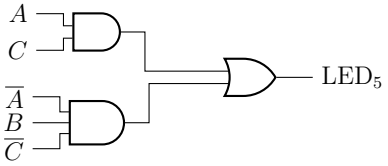
$$\text{LED}_4 = \overline{A}\,\overline{B} + A\overline{C} + BC$$



– **For LED$_5$:** [0.25]

$A\backslash BC$	00	01	11	10
0	0	0	0	1
1	0	1	1	0

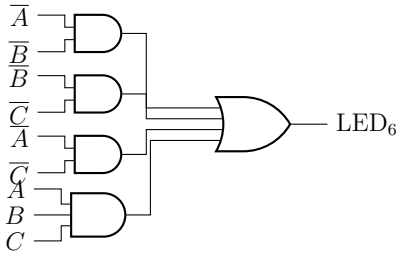
$$\text{LED}_5 = AC + \overline{A}B\overline{C}$$



– **For LED_6:** [0.25]

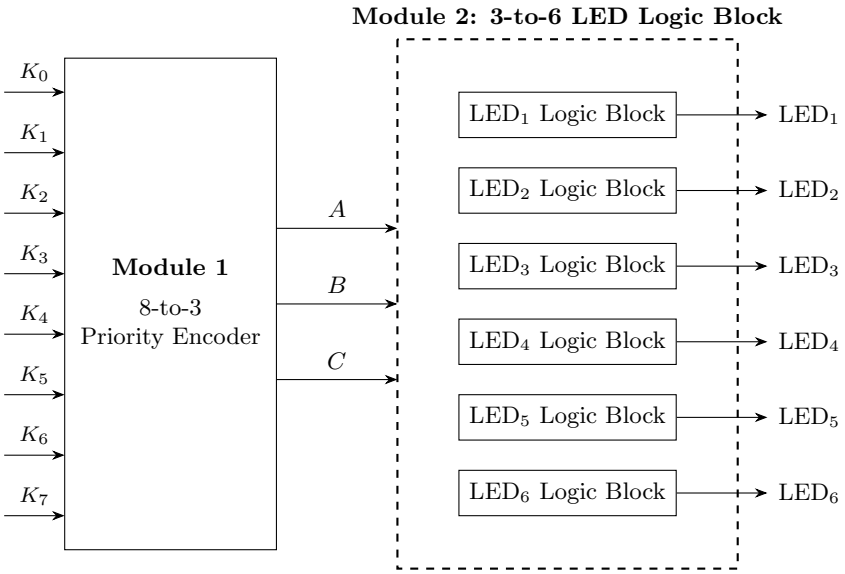
$A\backslash BC$	00	01	11	10
0	1	1	0	1
1	1	0	1	0

$$\text{LED}_6 = \overline{A}\,\overline{B} + \overline{B}\,\overline{C} + \overline{A}\,\overline{C} + ABC$$



d) [2]

High-Level Architecture Block Diagram



Method 2: Priority Encoder + 8:1 Multiplexers (MUX)

c) [4]

Module 1: 8-to-3 Priority Encoder [1.5]

– **Inputs:** $K_0, K_1, K_2, K_3, K_4, K_5, K_6, K_7$ [0.5]

– **Outputs:** A, B, C [0.5]

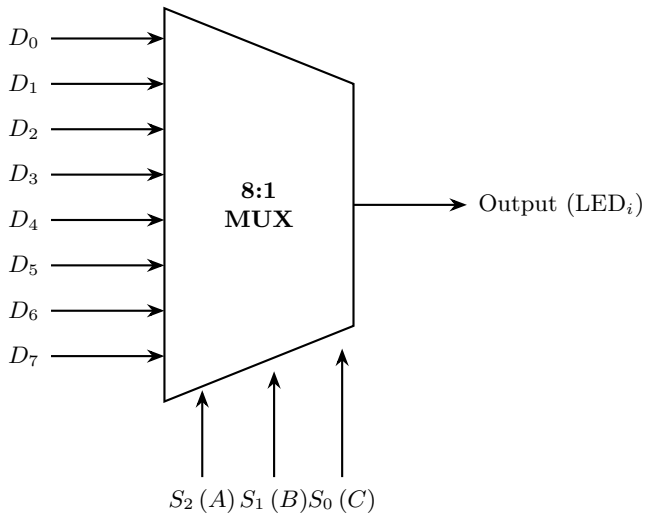
[0.5]

K_7	K_6	K_5	K_4	K_3	K_2	K_1	K_0	A	B	C
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	x	0	0	1
0	0	0	0	0	1	x	x	0	1	0
0	0	0	0	1	x	x	x	0	1	1
0	0	0	1	x	x	x	x	1	0	0
0	0	1	x	x	x	x	x	1	0	1
0	1	x	x	x	x	x	x	1	1	0
1	x	x	x	x	x	x	x	1	1	1

Module 2: Bank of Six 8:1 Multiplexers (MUX) [2.5]

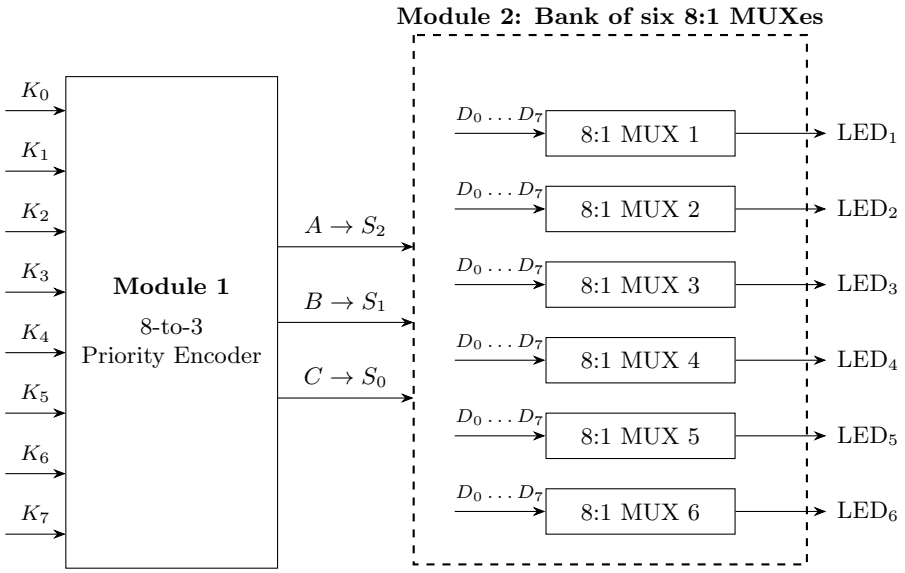
- **Inputs:** [0.5]
  - Select lines S_2, S_1, S_0 connected to Encoder outputs A, B, C.
  - Data inputs $D_0 \dots D_7$ tied to fixed logic levels (`1` or `0`) based on the LED requirements.
- **Outputs:** [0.5]
  - Six MUX outputs, each driving one LED ($\text{LED}_1 \dots \text{LED}_6$).
- **Operation:** The 3-bit encoder output selects which data pin (D_0 to D_7) connects to the output. The data pins for each MUX are hardwired to match its LED's column in the truth table. [0.5]

Data Input Mapping for each MUX: [1]



MUX Output	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
$\text{MUX}_1 \rightarrow \text{LED}_1$	1	0	1	1	0	1	1	0
$\text{MUX}_2 \rightarrow \text{LED}_2$	0	0	0	1	1	0	1	0
$\text{MUX}_3 \rightarrow \text{LED}_3$	0	1	0	0	0	0	1	1
$\text{MUX}_4 \rightarrow \text{LED}_4$	1	1	0	1	1	0	1	1
$\text{MUX}_5 \rightarrow \text{LED}_5$	0	0	1	0	0	1	0	1
$\text{MUX}_6 \rightarrow \text{LED}_6$	1	1	1	0	1	0	0	1

d) [2]



Method 3: Direct OR-Gate Matrix

a) The most obvious way to design this is to map the keys directly to the LEDs. Since each key activates a known pattern of LEDs, we can simply drive each LED with a single multi-input OR gate connected to the specific keys that require it to be ON. [1]

b) Since only one key is pressed at a time, there are actually no major functional flaws with this approach. The only downside is that the hardware wiring complexity is very high, requiring a lot of physical connections to the OR gates. [1]

c) [4]

Module: Multi-Input OR-Gate Block

- **Inputs:** $K_0, K_1, K_2, K_3, K_4, K_5, K_6, K_7$ [0.5]
- **Outputs:** $\text{LED}_1, \text{LED}_2, \text{LED}_3, \text{LED}_4, \text{LED}_5, \text{LED}_6$ [0.5]

Truth Table (System Mapping): [0.5]

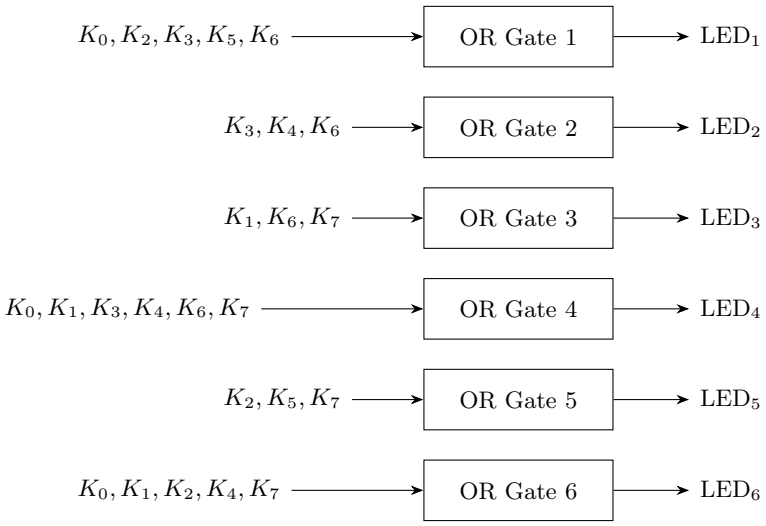
K_7	K_6	K_5	K_4	K_3	K_2	K_1	K_0	LED_1	LED_2	LED_3	LED_4	LED_5	LED_6
0	0	0	0	0	0	0	1	1	0	0	1	0	1
0	0	0	0	0	0	1	0	0	0	1	1	0	1
0	0	0	0	0	1	0	0	1	0	0	0	1	1
0	0	0	0	1	0	0	0	1	1	0	1	0	0
0	0	0	1	0	0	0	0	0	1	0	1	0	1
0	0	1	0	0	0	0	0	1	0	0	0	1	0
0	1	0	0	0	0	0	0	1	1	1	1	0	0
1	0	0	0	0	0	0	0	0	0	1	1	1	1

Logic Equations (Sum of Active Keys): [2.5]

- $\text{LED}_1 = K_0 + K_2 + K_3 + K_5 + K_6$
- $\text{LED}_2 = K_3 + K_4 + K_6$

- $\text{LED}_3 = K_1 + K_6 + K_7$
- $\text{LED}_4 = K_0 + K_1 + K_3 + K_4 + K_6 + K_7$
- $\text{LED}_5 = K_2 + K_5 + K_7$
- $\text{LED}_6 = K_0 + K_1 + K_2 + K_4 + K_7$

d) [2]



General Grading Note for Q5:

- **Valid Methods:** Any valid combinational circuit design(e.g., normal encoder etc) that correctly maps the 8 keys to the 6 LEDs is allowed and should be awarded full marks for the design section.
- **Completeness:** To get full credit, the student must have answered all sub-parts (a, b, c, and d) corresponding to their chosen method.
- **Addressing Part (b):** For the "issue/flaw" question (part b), the student is expected to at least attempt to address the downside (e.g., wiring complexity, 256-cell K-maps, hardware bulk). Award the mark if they reasonably address the problem.